

Unit 2: How Machines Learn

Study notes · Introduction to AI · Brightpath AI Academy, Bengaluru

2.1 Data, Features and Labels

A model learns from examples. Each example has **features** (the inputs) and a **label** (the answer we want). In Meera's mango project, the dataset has **240 mangoes**. The features are **weight in grams** and a **colour score from 1 to 10**; the label is **ripe or unripe**. The model never sees the word "ripe"; it only sees numbers and learns which numbers go with which label.

2.2 Loss Functions

A loss function measures how wrong a prediction is. For prices we often use **squared error**: $(\text{prediction} - \text{actual})^2$. Kiran's flat in **Indiranagar** was predicted at **Rs 82 lakh** but sold for **Rs 90 lakh**. The error is 8, so the squared error is **64**. Squaring makes big mistakes count much more than small ones. Training means finding weights that make the average loss as small as possible.

2.3 Gradient Descent

Gradient descent finds good weights step by step. Think of **Arjun walking down Nandi Hills in thick fog**: he cannot see the bottom, so he feels which way the ground slopes and takes a small step downhill. The slope is the **gradient**; the step size is the **learning rate**.

Rule: new weight = old weight – learning rate × gradient. Example: weight **4.0**, gradient **2.5**, learning rate **0.2** → new weight = $4.0 - 0.5 = 3.5$. Too large a learning rate overshoots the valley; too small takes forever.

2.4 Overfitting and Generalisation

A model **overfits** when it memorises the training data, noise included, instead of the pattern. Our model **MangoNet-7** scored **99% on training data but only 61% on new mangoes**. It is like **Tanvi**, who memorised last year's question paper word for word and struggled when the questions changed. Fixes: more data, a simpler model, **regularisation**, and always checking on a separate **validation set**.

2.5 Neural Networks

A neuron multiplies each input by a weight, adds a bias, and passes the total through an **activation function**. With inputs **(2, 1, 3)**, weights **(0.5, -1, 0.25)** and bias **0.5**: $1 - 1 + 0.75 + 0.5 = 1.25$. With **ReLU** (negative $\rightarrow 0$, positive unchanged) the output is **1.25**. Our example network has **3 inputs, one hidden layer of 4 neurons**, and 1 output. Layers of neurons let a network learn curved, complex patterns.

2.6 Attention and Transformers

Words change meaning with context. In "**The bank of the Kaveri was flooded**", the word "bank" should look at "Kaveri" to know it means a riverbank. **Attention** gives each word a score for every other word; here "bank" pays **0.71** of its attention to "Kaveri". Transformers stack many attention layers, which is how large language models read whole paragraphs at once.

Revision list

- Features are inputs; labels are answers.
- Loss measures error; training lowers it.
- Gradient descent: step against the slope, size = learning rate.
- Overfitting = memorising; check on validation data.
- Neuron = weighted sum + bias $\rightarrow$ activation.
- Attention lets each word look at the words that matter.